# Artificial_intelligence_assignment1_report_202
정현도

colab link : https://drive.google.com/file/d/1SYQeZLg9sJwkV8t-oF8bO1B98cqJAoye/view?usp=sharing

## Problem #1: Preparation of VOC Datasets and Visualization

### a) Prepare the VOC datasets

- **Augmenter**

**logic :**

Given an input image and its corresponding annotations (bounding box information), the image is horizontally flipped based on a probability condition. The x-coordinates of the bounding boxes are then updated to x= width - x.

**Troubleshooting:**

A precise understanding of tensor slicing and indexing was required.

- **Normalizer**

**logic:**

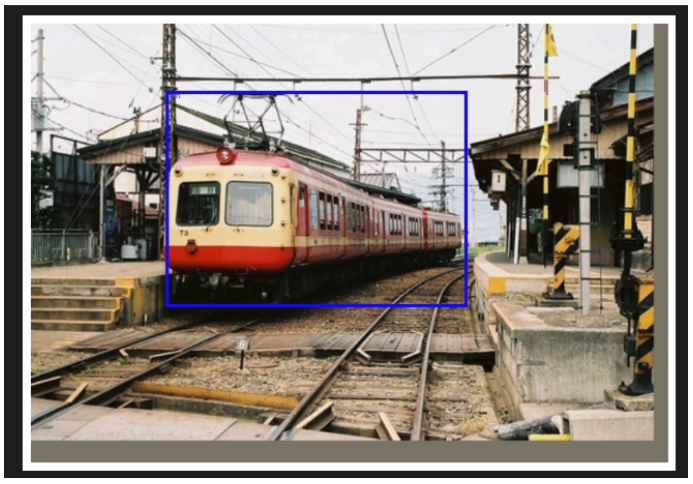
The image and annotations are retrieved from the sample dictionary. The image is then normalized using the following formula : image = (image - self.mean) / self.std

### b) Implement the VOCDataset class

**logic :**

The data is loaded and the structure of the dataset is analyzed in order to perform preprocessing that facilitates effective model training.

### c) Visualize images using the prepared VOCDataset instance.



*The augmenter was correctly implemented to ensure that both the annotations and the image are horizontally flipped in a consistent manner.*

## d) We use a collater() function to create the VOC Dataloader

**Why:**

The VOC dataset contains images of varying sizes and a different number of bounding boxes per image. Since deep learning models require inputs to be batched into tensors of uniform shape, the `collater()` function is responsible for padding each sample to the same size, allowing them to be assembled into a single batch tensor.

**Logic:**

To standardize image sizes, an empty tensor is created using `torch.zeros`, and each image is copied into its respective position. Since image sizes differ, it is important to initialize the empty tensor based on the maximum dimensions among the images.

## e) Complete the collater() function and initialize the Dataloaders using it

```
BATCH_SIZE = 2

##############################################################################
# TODO: Initialize dataloader_train and dataloader_val with collater()  #
##############################################################################
# [START]

sampler = AspectRatioBasedSampler(dataset_train, batch_size=BATCH_SIZE, drop_last=False)



# Train DataLoader
dataloader_train = DataLoader(
    dataset_train,
    batch_sampler=sampler,
    pin_memory=True,
```

```
    persistent_workers=True,
    collate_fn=collater
)

# Validation DataLoader
sampler_val = AspectRatioBasedSampler(dataset_val, batch_size=1, drop_last=False)
dataloader_val = DataLoader(
    dataset_val,
    batch_sampler=sampler_val,
    pin_memory=True,
    persistent_workers=True,
    collate_fn=collater
)
```

**Logic:**

The custom `collater` function was utilized, and a batch size of 1 was found to yield the fastest training speed on the current GPU.

---

# Problem #2: Building Models and Anchors

## a) Implement the architecture of the Feature Pyramid Network

**Concept:**

Feature Pyramid Network (FPN) receives intermediate outputs from the backbone network—specifically C3, C4, and C5—and generates multi-scale feature maps, namely P3, P4, P5, P6, and P7.

**logic:**

**Case: P5, P4, P3**

To construct feature maps at different pyramid levels:

- A 1×1 convolution is applied to adjust the channel dimensions to the desired feature size.
- The output from the previous level is upsampled and merged with the current level's feature map (P5 is excluded from this step as it does not have a higher-level input).
- A 3×3 convolution is then applied to refine the merged features and produce the final output for each level.

**Case: P6, P7**

- P6 is generated by applying a 3×3 convolution with stride 2 directly on C5.
- P7 is then obtained by applying a ReLU activation followed by another 3×3 convolution with stride 2 on P6.

## (b) Implement the forward pass of the Feature Pyramid Network

```
class PyramidFeatures(nn.Module):
    def __init__(self, C3_size, C4_size, C5_size, feature_size=256):

        super(PyramidFeatures, self).__init__()



        self.P5_1 = nn.Conv2d(C5_size, feature_size, kernel_size=1, stride=1, padding=0)
```

```
        self.P5_upsampled = nn.Upsample(scale_factor=2, mode='nearest')

        self.P5_2 = nn.Conv2d(feature_size, feature_size, kernel_size=3, stride=1, padding=1)


        self.P4_1 = nn.Conv2d(C4_size, feature_size, kernel_size=1, stride=1, padding=0)

        self.P4_upsampled = nn.Upsample(scale_factor=2, mode='nearest')

        self.P4_2 = nn.Conv2d(feature_size, feature_size, kernel_size=3, stride=1, padding=1)


        self.P3_1 = nn.Conv2d(C3_size, feature_size, kernel_size=1, stride=1, padding=0)

        self.P3_2 = nn.Conv2d(feature_size, feature_size, kernel_size=3, stride=1, padding=1)


        self.P6 = nn.Conv2d(C5_size, feature_size, kernel_size=3, stride=2, padding=1)


        self.P7_1 = nn.ReLU()

        self.P7_2 = nn.Conv2d(feature_size, feature_size, kernel_size=3, stride=2, padding=1)

    def forward(self, inputs):


        C3, C4, C5 = inputs


        P5_x = self.P5_1(C5)
        P5_upsampled_x = self.P5_upsampled(P5_x)
        P5_x = self.P5_2(P5_x)


        P4_x = self.P4_1(C4)
        P4_x = P4_x + P5_upsampled_x
        P4_upsampled_x = self.P4_upsampled(P4_x)
        P4_x = self.P4_2(P4_x)


        P3_x = self.P3_1(C3)
        P3_x = P3_x + P4_upsampled_x
        P3_x = self.P3_2(P3_x)


        P6_x = self.P6(C5)


        P7_x = self.P7_1(P6_x)
        P7_x = self.P7_2(P7_x)

        return [P3_x, P4_x, P5_x, P6_x, P7_x]
```

## c) Implement the architecture and forward pass of the Regression Model (box subnet)

**Concept:**

The regression model predicts bounding box adjustment values—(dx, dy, dw, dh)—for each anchor based on the feature maps generated by the FPN.

**Logic:**

Four consecutive 3×3 convolutional layers, each followed by a ReLU activation, are used to progressively extract deeper features from the input feature map. A final output convolutional layer is applied, where the number of output channels is set to **num_anchors × 4** to predict four bounding box regression values for each anchor.

## (d) Implement the architecture and forward pass of the Classification Model (class subnet)

**Concept:**

The classification model predicts the probability of each object class for every anchor.

**Logic:**

Similar to the regression model, four 3×3 convolutional layers—each followed by a ReLU activation—are used to extract features from the input. A final output convolutional layer sets the number of output channels to **num_anchors × num_classes**. The Sigmoid function is then applied to convert the outputs into probabilities between 0 and 1.

## (e) Implement the BBoxTransform, Anchors classes and generate_anchors(), shift() functions

**BBoxTransform:**

This module takes the regression model's predicted values (Δx, Δy, Δw, Δh) and converts them into actual bounding box coordinates. Since the predicted values are normalized, they are first rescaled to their original scale. Then, using the center coordinates and dimensions of the anchor boxes, the final bounding box coordinates are computed.

**Anchors:**

Anchor boxes are generated for each level of the Feature Pyramid Network using the `generate_anchors()` function. These base anchors are then shifted to align with the center of each cell location on the corresponding feature maps using the `shift()` function.

**generate_anchors():**

Based on a specified base size, this function generates multiple anchors using combinations of aspect ratios and scales. In this implementation, a base size of 16, aspect ratios of [0.5, 1, 2], and scales of [1, 1.3, 1.6] were used, resulting in 9 anchors per location.

**shift():**

For each level of the feature pyramid, base anchors are replicated and shifted to align with the center of every cell in the feature map, effectively covering the entire image. Since the resolution of feature maps decreases by a factor of 2×2^x2x compared to the original image, it is critical to apply the appropriate stride value when shifting anchors.

---

# Problem #3: Training Models

## (a) Train RetinaNet using BCE loss. Complete the training loop.

- **Data Preparation:**

  Images and annotations loaded through the `DataLoader` are transferred to the GPU.

- **Forward Pass:**

  A tuple of (images, annotations) is passed into the model to compute the classification and regression losses.

- **Total Loss Calculation:**

  The total loss is computed by summing the classification loss and the regression loss.

- **Backward Pass:**

  Gradients are calculated by performing backpropagation on the total loss.

- **Parameter Update:**

  Model parameters are updated using the optimizer.

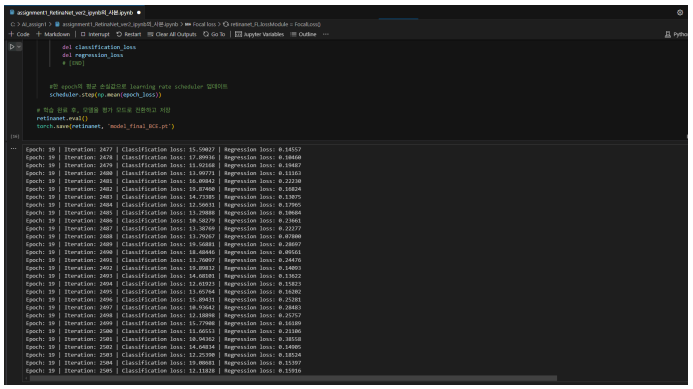
- **Loss Monitoring and Logging:**

  The loss from each iteration is recorded and printed for monitoring purposes.

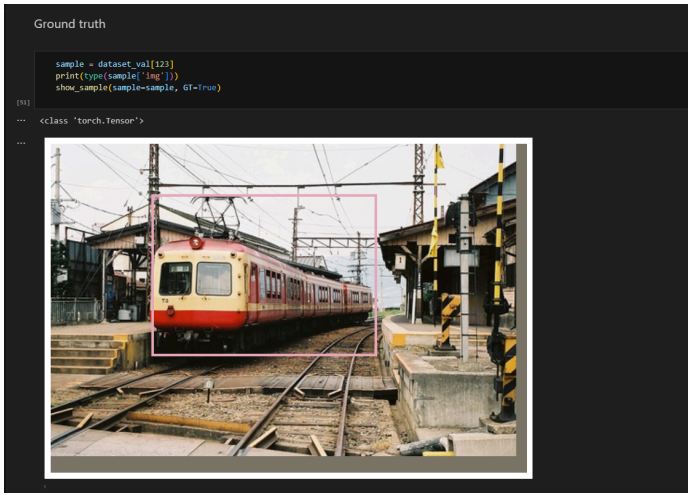
- **Learning Rate Scheduler Update:**

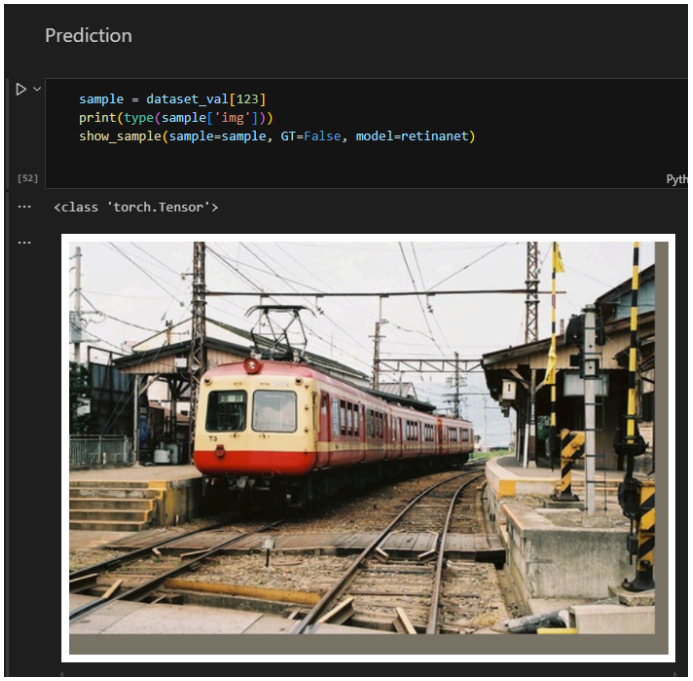
  At the end of each epoch, the learning rate is adjusted based on the average loss using a learning rate scheduler.

## (b) Visualize the test set using the trained RetinaNet and compare it with its ground truth.




*Due to execution limitations on Google Colab, the model was trained locally laptop using VS Code.*






*The BCE loss model may suffer performance degradation when the dataset contains a large number of background anchors. This is because the loss can be dominated by background anchors, which vastly outnumber foreground ones during training.*

**In the end, anchors with low confidence scores are filtered out and therefore not visualized in the `show_sample` function.**

## (c) Train RetinaNet using focal loss. Complete the training loop.

- **Data Preparation:**

  Images and annotations loaded through the `DataLoader` are transferred to the GPU.
- **Forward Pass:**

  A tuple of (images, annotations) is passed into the model to compute the classification and regression losses.
- **Total Loss Calculation:**

  The total loss is computed by summing the classification loss and the regression loss.
- **Backward Pass:**

  Gradients are calculated by performing backpropagation on the total loss.
- **Parameter Update:**

  Model parameters are updated using the optimizer.
- **Loss Monitoring and Logging:**

  The loss from each iteration is recorded and printed for monitoring purposes.
- **Learning Rate Scheduler Update:**

  At the end of each epoch, the learning rate is adjusted based on the average loss using a learning rate scheduler.

## (d) Visualize the test set using the trained RetinaNet and compare it with its ground truth.

```
# 9. (옵션) 메모리 관리를 위해, 사용한 손실 변수들을 삭제합니다
del classification_loss
del regression_loss
# [END]

scheduler.step(np.mean(epoch_loss))

retinanet_FL.eval()
torch.save(retinanet_FL, 'model_final_FL.pt')
```

[20] ✓ 524m 41.5s Python

```
Num training images: 5011
C:\Users\정현도\AppData\Local\Temp\ipykernel_21136\3448438985.py:82: UserWarning: To copy
  annots = torch.tensor(annots, dtype=torch.float32)
Epoch: 0 | Iteration: 0 | Classification loss: 1.13585 | Regression loss: 1.05392
Epoch: 0 | Iteration: 1 | Classification loss: 1.19003 | Regression loss: 0.94370
Epoch: 0 | Iteration: 2 | Classification loss: 1.20282 | Regression loss: 0.97600
Epoch: 0 | Iteration: 3 | Classification loss: 1.18601 | Regression loss: 0.98024
Epoch: 0 | Iteration: 4 | Classification loss: 1.16117 | Regression loss: 1.07524
Epoch: 0 | Iteration: 5 | Classification loss: 1.20276 | Regression loss: 1.01019
Epoch: 0 | Iteration: 6 | Classification loss: 1.14795 | Regression loss: 1.05900
Epoch: 0 | Iteration: 7 | Classification loss: 1.12109 | Regression loss: 1.04536
Epoch: 0 | Iteration: 8 | Classification loss: 1.16070 | Regression loss: 1.03559
Epoch: 0 | Iteration: 9 | Classification loss: 1.25994 | Regression loss: 1.00214
Epoch: 0 | Iteration: 10 | Classification loss: 1.22176 | Regression loss: 1.03796
Epoch: 0 | Iteration: 11 | Classification loss: 1.17222 | Regression loss: 0.98366
Epoch: 0 | Iteration: 12 | Classification loss: 1.16572 | Regression loss: 1.06718
Epoch: 0 | Iteration: 13 | Classification loss: 1.18740 | Regression loss: 1.05675
Epoch: 0 | Iteration: 14 | Classification loss: 1.13713 | Regression loss: 1.04089
Epoch: 0 | Iteration: 15 | Classification loss: 1.15437 | Regression loss: 1.08145
Epoch: 0 | Iteration: 16 | Classification loss: 1.16978 | Regression loss: 1.06094
Epoch: 0 | Iteration: 17 | Classification loss: 1.16147 | Regression loss: 1.06764
Epoch: 0 | Iteration: 18 | Classification loss: 1.14833 | Regression loss: 1.02433
Epoch: 0 | Iteration: 19 | Classification loss: 1.16660 | Regression loss: 1.10974
Epoch: 0 | Iteration: 20 | Classification loss: 1.20042 | Regression loss: 0.98533
Epoch: 0 | Iteration: 21 | Classification loss: 1.16292 | Regression loss: 1.03477
Epoch: 0 | Iteration: 22 | Classification loss: 1.12738 | Regression loss: 1.04238
Epoch: 0 | Iteration: 23 | Classification loss: 1.08608 | Regression loss: 1.06416
Epoch: 0 | Iteration: 24 | Classification loss: 1.15623 | Regression loss: 1.02604
...
Epoch: 19 | Iteration: 2502 | Classification loss: 0.11308 | Regression loss: 0.28980
Epoch: 19 | Iteration: 2503 | Classification loss: 0.03123 | Regression loss: 0.16020
Epoch: 19 | Iteration: 2504 | Classification loss: 0.04952 | Regression loss: 0.23896
Epoch: 19 | Iteration: 2505 | Classification loss: 0.01479 | Regression loss: 0.07998
```

*Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...*

```
sample = dataset_val[123]
print(type(sample['img']))
show_sample(sample=sample, GT=False, model=retinanet_FL)
```

```
<class 'torch.Tensor'>
classification device: cuda:0
transformed_anchors device: cuda:0
max class prob: tensor(0.2561, device='cuda:0')
```

## (e) Describe the differences in the inference results of the models when using BCE loss versus focal loss.

- **Model Using BCE Loss:**

  Loss is computed uniformly across all anchors, which can cause the overall loss to be dominated by the large number of background anchors.

  As a result, the model may show reduced sensitivity to small objects and hard examples.

- **Model Using Focal Loss:**

  Focal Loss reduces the contribution of well-classified anchors and emphasizes harder examples during training.

This makes the model more robust to class imbalance and improves performance on small objects and challenging cases.